

TECHNICAL ASSESSMENT PROJECT

Senior Developer.

Building with Claude on the riivo stack

WELCOME

Let's do this.

At riivo we pride ourselves on an exceptional team delivering exceptional solutions, and we do not compromise on that.

The way we build has changed. Our developers work hand in hand with Claude to ship real software on a modern stack, and at the senior level we trust you to lead that work on client projects with little oversight. You turn a client's problem into a sound product and a working build, and you own the decisions along the way.

This project is deliberately open, because that reflects the real job. We are excited to see how you approach it. Good luck.

TOP TIP

There is no single right answer here. What we are looking at is the quality of your thinking: what you chose to build, how you modelled it, and how well you can stand behind the decisions you made.

THE BRIEF

What we want you to build.

THE STARTING POINT

Build a web app that helps someone know when satellites will be visible or contactable from their location.

You are going to build a real application on our stack: Next.js, Supabase and Vercel, deployed live. The theme is satellite tracking, and if the subject is new to you, working out an unfamiliar domain is part of what we are assessing.

Unlike the more junior assessments, we are not going to hand you a list of acceptance criteria. Instead we give you a business problem, the way a client would, along with a few firm constraints. The product and architecture decisions are yours to make. Treat the brief on the next slides as the opening of a client conversation, and show us how you would respond to it.

What a strong senior shows us.



Turns a problem into a design

Takes a high level business need and proposes a sound, well scoped solution with a clear data model.



Models shared state well

Handles multiple users coordinating around the same thing, with a design that does not fall apart.



Builds a reliable pipeline

An async alerting flow that is dependable, batched, and considerate of the people receiving it.



Owns and defends the call

Integrates several APIs under real constraints, and can justify every significant decision made.

The client brief.

"Our users are amateur radio operators and satellite spotters. They gather in groups around good passes, the moments a satellite is high enough and well placed to actually work or see. Right now they check several different tools and message each other by hand, and they keep missing passes. Build them something that fixes this."

That is the brief. The constraints: real users with accounts, it must genuinely notify people, it must run against live satellite data, and it must hold up under real use.

Everything else is yours to decide: what to build, how to model it, what a good pass means, how people coordinate, and how alerts stay useful rather than noisy. Write it down and defend it.

YOUR TASKS

Core first, then stretch if you like.

Task	Name	What it covers	Est.
Core 1	Interpret and design	Decide what to build, write a short design note: data model, scope, what good pass and coordinate mean, how you will visualise it	1-2h
Core 2	Foundation and model	Stand up the app, deploy, accounts, and implement your data model including any shared state	2-3h
Core 3	Live data integration	Integrate the tracking APIs under real constraints, with caching and failure handling, across users and objects	2h
Core 4	Coordination feature	Whatever you decided coordinate means: shared lists, group subscriptions, and so on, built and working	2h
Stretch A	Visualisation	Your chosen visual of passes for a group: ground track map, sky plot, shared pass calendar, or better	+2-3h
Stretch B	Alerting pipeline	Scheduled notifications by email or SMS, batched and de-duplicated so a popular pass does not spam a group	+2-3h
Stretch C	Make it hold up	Real use robustness and hardening under load and edge cases	+1-2h

The core is the expected ask, roughly five to seven hours. The design note in Core 1 is where we see your thinking, so do not skip it. Stretch tasks are optional upside.

The APIs you will use.

You choose your APIs from this shortlist. We expect you to combine several to meet the brief.

- Tracking: N2YO (key) or Where the ISS at (no key) for positions and passes.
- Geocoding: Open-Meteo Geocoding or Nominatim (no key).
- Daylight and visibility: sunrise-sunset.org (no key).
- Alerts: Resend (email) or Twilio (SMS), both free to start.

All of these are free. Where a key is needed, the free tier is enough. The map route, if you take it, can use Leaflet with OpenStreetMap tiles or react-simple-maps, both free.

Send us these five things.



1. A GitHub repository

Public or shared with us. We look at how it was set up and how it came together.



2. A live, working link

Your app on Vercel and Supabase. Open it yourself before submitting and confirm it runs.



3. A test account

A known login, already set up with a couple of saved locations and satellites.



4. A README

In the repo root. Headings: Live URL, Test account, APIs used, Running locally, Assumptions.



5. Your assumptions

Under that heading, the decisions you made where the brief was silent, and why.

Keep it simple. If something is half finished, tell us in the README rather than hiding it. We would always rather see honest, working software than a demo that falls over when we click around.

ONE LAST THING

Skills can be taught. Attitude cannot. We hire people, not certificates.

Just do your best. We are good at spotting potential, and we want to see how you think.